CS781 Capstone Project: In-House Legal Intelligence System

Vivek Chandwani
241183
vivekc24@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

This project builds a self-hosted legal NLP system for confidential case documents. Its seven tasks cover legal continuation, named entity recognition, rhetorical role prediction, judgment prediction, statute identification, abstractive summarization and prior case retrieval. Six foundation models were trained from scratch on official Acts, High Court, Supreme Court and Tribunal data. The best Phase-1 model was a 122.12M-parameter decoder with grouped-query attention and alternating local and full attention. Its checkpoint family was adapted through full fine-tuning, low-rank methods and knowledge distillation. After Phase 1, Model 7 extended the project with a novel legal-domain application of DiLoCo. Three replicas trained on different legal distributions and periodically merged their weights. This compact 110.10M-parameter model received approximately 4B training tokens and achieved the best primary scores in P3–P7. The results highlight both effective foundation transfer and the value of domain-specialized training under limited compute.

1 Competition Result

Codabench username: v_241183 **Roll number:** 241183

Table 1: Official phase scores and ranks.

Phase	Task	Primary metric (value)	Rank	Valid submissions
1	Legal LM Pretraining	0.838973	20	15
2	Legal NER	0.597164	21	11
3	Rhetorical role prediction	0.525001	42	10
4	Court judgment prediction	0.813447	12	4
5	Legal statute identification	0.412709	28	6
6	Abstractive summarization	0.850100	23	8
7	Prior Case Retrieval (PCR)	0.4743059657	18	14

Total valid Codabench submissions across all phases: 68
Date ranks were recorded: 09/11/2026
I met the minimum submissions requirement in all the 7 phases.

2 Introduction and Problem Description

Legal documents contain sensitive facts, long arguments and references to earlier cases. A self-hosted system can support their analysis while keeping documents within the organization. This project develops seven legal NLP tasks based on the CS781 competition and the IL-TUR benchmark [1, 2].

Phase 1 generates a 128-word continuation from a legal prompt. BERTScore F1 is the primary metric. Phase 2 returns character spans and entity types for a judgment. Strict macro-F1 requires the type and both boundaries to match. Phase 3 assigns one of 13 rhetorical roles to each sentence. Phase 4 predicts a binary judgment outcome. Both use macro-F1. Phase 5 predicts a set of applicable statute IDs and uses macro-F1, micro-F1 and subset accuracy. Phase 6 generates a legal summary and uses BERTScore F1 with ROUGE-1 and ROUGE-L. Phase 7 ranks exactly seven unique prior-case IDs from the query's candidate pool. Its micro-F1@7 compares those predictions with the full relevant set.

The foundation models had to contain between 50M and 300M parameters. They were trained from random initialization using official legal data. Limited GPU memory also restricted document coverage. I compared architectures, training exposure, adaptation methods, document representations and decoding. Model 4's Phase-1 result motivated Model 7, which was trained after Phase 1 ended. Its domain-specific replicas and periodic merging form the project's main methodological contribution. The project covers model training and task predictions. Deployment and human legal evaluation remain future work.

3 Data Analysis and Preprocessing

3.1 Sources and official splits

The official CS781 dataset contains Acts, High Court judgments, Supreme Court judgments and Tribunal decisions [1]. The 10% pretraining sample contains 348,521 documents. Table 2 summarizes the task datasets. For NER, eight documents from fold 2 formed the validation set. Summarization used a document-level validation split containing 10% of the training pairs.

Table 2: Dataset sizes and validation splits. Training counts precede local holdout selection. Retrieval counts give queries followed by candidate cases.

Phase	Training	Validation	Test
1	Four legal corpora	Document holdout	1,000 prompts
2	70 documents in two folds	8 held out locally	35 documents
3	CL 40 and IT 40	CL 5 and IT 5	CL 5 and IT 5
4	37,387	1,988	1,517
5	42,750	10,181	13,019
6	7,030 pairs	10% held out locally	100
7	827 and 4,320	118 and 1,023	237 and 1,727

3.2 Corpus statistics that actually informed design

I used 40% Supreme Court, 30% High Courts, 15% Acts and 15% Tribunals to prevent the largest sources from dominating training (Figure 1). Median length is 2,727 words for Acts and 265 for High Courts. Equal-token samples contain 14,484 Act word types and 26,098 Tribunal types. These domain differences motivated Model 7's replicas.

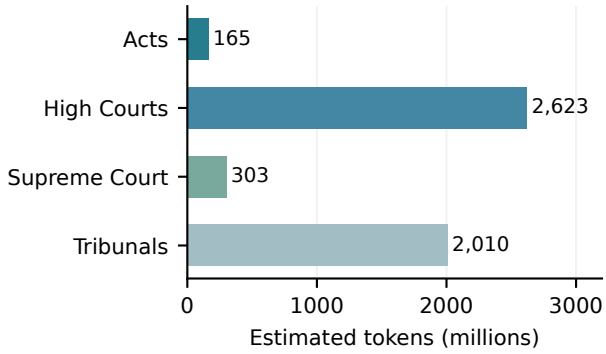


Figure 1: Estimated pretraining tokens by source. Estimates use a 10% corpus sample.

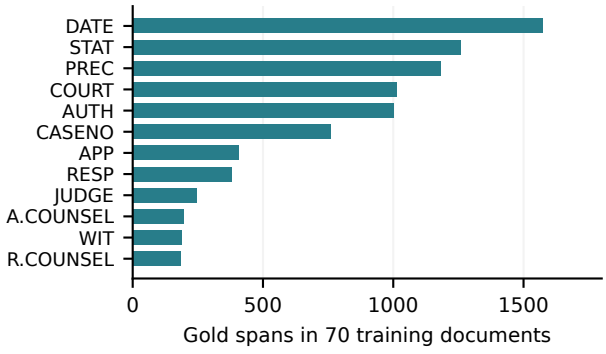


Figure 2: Training entity frequencies. Rare types receive much less supervision.

NER contains 8,393 spans across 12 types (Figure 2). No overlap or nesting was detected. The 107 adjacent same-type spans favor BIO. Statute spans reached 233 characters compared with 28 for dates. Overlap helps preserve complete entities at boundaries.

Corporate Law (CL) and Information Technology (IT) contain 10,593 and 6,252 training sentences. Statute accounts for 3.46% of CL roles and 11.71% of IT roles. Dissent occurs 284 times in two CL documents. PrecedentOverruled has only three IT examples. The 95th percentile sentence length is 72 words for CL and 84 for IT. Domain tags and class weighting address these differences.

Judgment outcomes are split 59% to 41% in training. Median length is 2,192 words. The final 512 words contain 46.1% of the last detected outcome phrases, compared with 26.6% in the first 512 words. This supports retaining document endings. Statute identification has 100 labels and averages 3.7 labels per case. The ten most frequent labels account for 41.2% of assignments, although every label has at least 100 training examples. Median label count is three, and the 95th percentile is eight.

For summarization, median document and summary lengths are 3,128.5 and 638 words. Median compression is 4.6, and 21.8% of summary bigrams are new. Retrieval median lengths are 4,508 words for queries and 3,632 for candidates. Mean relevant-set size is 6.42 in training and 8.25 in validation. Long documents require chunked encoding. Dates do not establish reliable temporal ordering, and court level is unclear for 58.4% of training queries. Date and court filters were therefore not used.

3.3 Preprocessing

Models 1 to 6 use a custom cased 32K byte-level BPE tokenizer that preserves Unicode and citation symbols. Model 7 uses 30,522 uncased WordPieces, which removes name capitalization. For NER, the original text stays unchanged. Token offsets exclude surrounding whitespace. Gold spans were mapped to BIO tags, and decoded tags were converted back to character intervals. Sentence order was preserved for rhetorical roles. Long documents were divided into overlapping windows. Retrieval used the same packing and special-token rules for queries and candidates.

4 Approach and Model Evolution

4.1 Phase-1 foundation model

I first compared several ways to model legal text. Model 1 used a causal Transformer with Plasticity-Adjusted Sampling. Model 2 used an encoder and decoder. Model 3 used bidirectional attention for the prompt and causal attention for the continuation. Models 5 and 6 increased model capacity.

Model 4 achieved the best Phase-1 BERTScore. It has 16 layers, hidden size 768, 12 query heads, two KV heads and an FFN size of 2,048. It uses RoPE, pre-RMSNorm and SwiGLU. Layers alternate between a 256-token causal window and full causal attention. Counting each tied weight once gives 122,118,912 parameters, within the required bound. Next-token training used 1,024-token blocks and 1.5B tokens. Generation used temperature 0.5, top-k 50, top-p 0.9 and repetition penalty 1.1. It targeted 128 words with a maximum of 384 new tokens. The larger Model 6 did not improve BERTScore.

After Phase 1 ended, I trained Model 7 using a domain-specific application of DiLoCo [25]. Its 110.10M-parameter BERT-style prefix model has 12 layers, hidden size 768, 12 heads, FFN size 3,072, post-LayerNorm, GELU and 512 learned positions. Three replicas start from the same random weights. One trains on Supreme Court judgments, one on High Court judgments, and one on Acts and Tribunals. Each runs 500 local AdamW steps before a Nesterov outer update merges the parameter changes. The merged weights start the next round, while each replica keeps its optimizer state. This allows domain-specific learning followed by regular consolidation. About 4B aggregate training tokens give 36.3 tokens per parameter. The resulting single backbone is adapted for P3–P7.

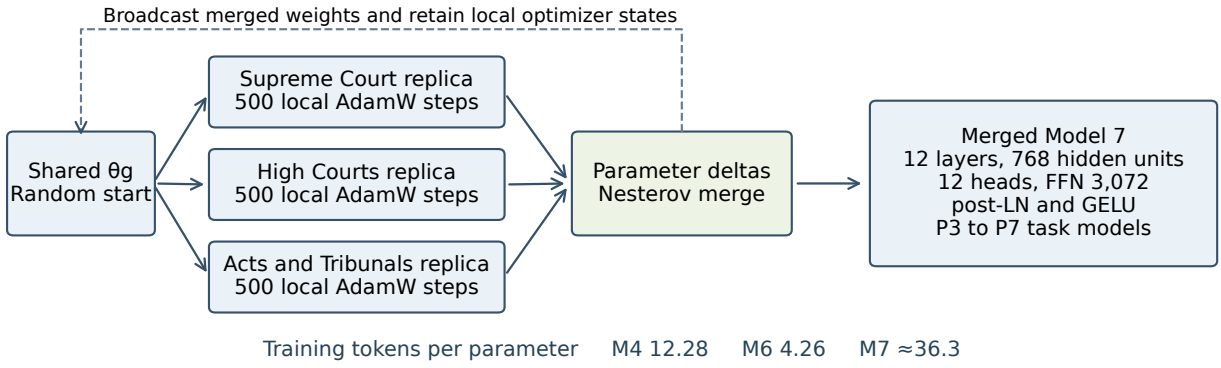


Figure 3: Model 7 training. Domain replicas learn separately and periodically merge into one backbone. Each later task uses its own adapted copy.

4.2 Transfer protocol (Phases 2–7)

Each task starts from a foundation checkpoint and adds a task head. Full fine-tuning updates all weights. LoRA [30] freezes the base matrix and adds $y = Wx + (\alpha/r)BAx$. LoRMA [31] instead applies the output-space update $y = (I + (\alpha/r)BA)Wx$. Both use rank 8 and $\alpha = 16$. Initializing B to zero preserves the starting predictions. Knowledge distillation [37] combines hard labels with teacher outputs and trains a student with fewer layers. Models 4 and 6 use bidirectional masks for downstream encoding. Model 7 uses its bidirectional prefix representation.

4.3 Phase 2 - Legal NER

A 25-tag BIO classifier and a CRF process 512-token windows with stride 256. Center-weighted averaging combines overlapping emissions before document-level Viterbi decoding. The resulting BIO path is mapped to character spans. This gives one flat tagging sequence without independent span thresholds. Duplicate spans are removed. Figure 2 lists the 12 entity types. A weighted cross-entropy alternative addresses imbalance. Model 6 with LoRA performs best. I also tested partial tuning, bottleneck adapters, prompts, prefixes and BitFit.

4.4 Phase 3 - Rhetorical role prediction

Each target sentence receives three preceding sentences, one following sentence, a domain tag and one of 20 position buckets. Context sentences are clipped to 48 tokens and targets to 200 tokens within a 384-token input. Target and input means feed an independent classifier without a document CRF. Joint CL and IT training uses inverse-square-root class weights. Model 7 with LoRMA performs best.

4.5 Phase 4 - Court judgment prediction

Six 384-token windows with stride 256 cover selected beginning, middle and ending passages. Masked token means form window vectors, which learned attention combines into a document vector. A binary head predicts the outcome. Class weights address the training imbalance. Validation macro-F1 selects the checkpoint. The decision threshold is tuned from 0.10 to 0.90 in steps of 0.01. Entity and role predictions are not used as features.

4.6 Phase 5 - Legal statute identification

Six windows feed label-wise attention and 100 independent sigmoid outputs. Weighted BCE uses square-root class ratios capped at 30. Thresholds are searched from 0.03 to 0.95 in steps of 0.02. Per-label thresholds combine 70% of the label optimum with 30% of the global threshold. Repeated validation splits choose global or per-label calibration. An argmax fallback returns at least one label.

4.7 Phase 6 - Abstractive summarization

Two foundation copies initialize an encoder and a causal decoder. New cross-attention connects the decoder to concatenated token states from four 256-token windows with stride 192. Training targets are capped at 256 tokens. Generation uses four beams, repeated-trigram blocking and length penalty 1.0. Sequence distillation mixes references and teacher summaries equally for training documents. Its student has six encoder layers and six decoder layers.

4.8 Phase 7 - Prior Case Retrieval (PCR)

A shared bi-encoder processes queries and candidates with six 384-token windows and stride 320. Long cases use two early, two middle and two late windows. Attention pooling produces normalized case vectors. InfoNCE uses temperature 0.05, in-batch negatives and one TF-IDF hard negative from a top-50 pool. Validation selects the dense and lexical score blend. Inference excludes the query itself and ranks seven candidates. Distillation combines InfoNCE with teacher score and embedding matching.

5 Experiments and Implementation Details

5.1 Compute and training protocol

Pretraining and NER ran on a Tesla T4 with 15.6 GB memory. The NER LoRA run completed 15 epochs in 40.4 minutes. Training used AdamW, mixed precision and clipping at 1.0. Pretraining checkpoints also saved optimizer state. Pretraining used 1,000 warmup steps and cosine decay in the final 10%. Task training used linear warmup and decay.

Table 3: Training configuration. Parameter counts are backbone sizes in millions. Epoch values are training limits.

Hyperparameter	P1	P2	P3	P4	P5	P6	P7
Best model	M4	M6 LoRA	M7 LoRMA	M7 Full	M7 Full	M7 Full	M7 Full
Backbone size	122.12	234.66	110.10	110.10	110.10	2×110.10	110.10
Window budget	1024×1	512	384×1	384×6	384×6	256×4	384×6
Backbone LR	$6 \cdot 10^{-4}$	$3 \cdot 10^{-4}$	$3 \cdot 10^{-4}$	$5 \cdot 10^{-5}$	$5 \cdot 10^{-5}$	$5 \cdot 10^{-5}$	$5 \cdot 10^{-5}$
Head LR	tied	$5 \cdot 10^{-4}$	$5 \cdot 10^{-4}$	$5 \cdot 10^{-4}$	$5 \cdot 10^{-4}$	10^{-4}	10^{-3}
Batch × accum.	16×4	8×1	16×1	1×8	1×8	1×8	6×1
Budget	1.5B tokens	50 epochs	8 epochs	5 epochs	5 epochs	6 epochs	20 epochs
Warmup	1,000 steps	6%	6%	8%	3%	10%	6%
Weight decay	0.1	0.01	0.01	0.01	0.01	0.01	0.01
Backbone dropout	0	0	0.1	0.1	0.1	0.1	0.1
Patience	val NLL	10	3	2	3	2	4
Prediction rule	sampling	CRF	argmax	dev tuned	dev tuned	4 beams	top 7

5.2 Local evaluation

NER used 62 training and eight validation documents. Different entity frequencies can explain the higher competition F1 of 0.597164. Selection used strict NER F1, mean domain role F1, classification macro-F1, summarization BERTScore and retrieval F1@7. Local summary BERTScore used DistilBERT and is not directly comparable with competition values.

Table 4: Strongest local NER variants. P and R denote strict macro precision and recall.

Method	P	R	F1
LoRA	**0.567**	0.530	**0.5315**
Distillation	0.5327	**0.5610**	0.5312
Pfeiffer adapter	0.5448	0.5368	0.5297

5.3 Ablations and controlled comparisons

With fixed NER logits, punctuation trimming raised macro-F1 from 0.5299 to 0.5315. Further period stripping reduced it to 0.5262. Prompt and prefix tuning reached 0.4172 and 0.3999. For fixed Model 4 weights, sampling gave approximately 0.8257 local BERTScore and greedy decoding gave 0.8125. Polishing did not replace the original model.

6 Results

Tables 5 to 11 compare the primary scores for all model variants. Bold values mark the best score in each phase. The Phase-1 table also highlights each secondary metric. P2 compares eight variants from two backbones. Table 1 gives the overall phase scores and ranks. Full secondary results are in Appendix A.

Model	BERTScore	ROUGE-L	BLEU-4
M1	0.831894	0.191080	0.043804
M2	0.828659	0.191965	0.040064
M3	0.827758	**0.194339**	**0.046634**
M4	**0.838973**	0.188266	0.038554
M5	0.833433	0.191161	0.043302
M6	0.833631	0.193479	0.044605

Table 5: P1: all scored foundations.

Model	Full	LoRA	LoRMA	KD
M4	0.507839	0.529345	0.525853	0.494830
M6	0.548211	**0.597164**	0.563143	0.496312

Table 6: P2: Strict Macro-F1.

Model	Full	LoRA	LoRMA	KD
M4	0.490116	0.509134	0.517230	0.489623
M6	0.499972	0.513336	0.522814	0.492541
M7	0.505800	0.513997	**0.525001**	0.493800

Table 7: P3: Macro-F1.

Model	Full	LoRA	LoRMA	KD
M4	0.793281	0.786215	0.781846	0.776592
M6	0.798526	0.791384	0.786927	0.781463
M7	**0.813447**	0.805862	0.800110	0.795173

Table 8: P4: Macro-F1.

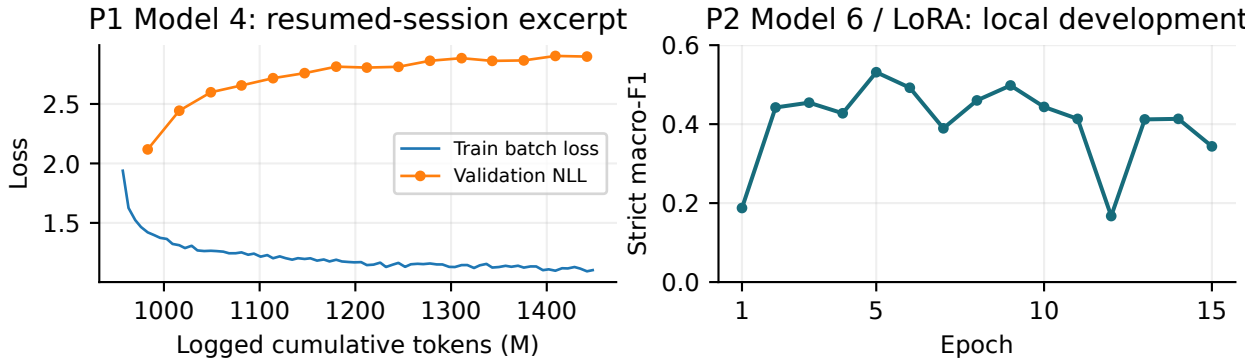


Figure 4: Learning curves for a Model 4 pretraining session and Model 6 NER training with LoRA.

Model 4 training loss fell while validation NLL rose. Lower training loss therefore did not guarantee better validation performance. NER reached its best local macro-F1 of 0.5315 at epoch 5. Later epochs did not recover that value. These curves support validation-based checkpoint selection.

Model	Full	LoRA	LoRMA	KD
M4	0.406448	0.397638	0.396284	0.391752
M6	0.412469	0.403817	0.402614	0.397926
M7	**0.412709**	0.404459	0.403452	0.398731

Table 9: P5: Macro-F1.

Model	Full	LoRA	LoRMA	KD
M4	0.837233	0.826392	0.831684	0.821853
M6	0.843672	0.832547	0.839214	0.827541
M7	**0.850100**	0.835926	0.845170	0.826851

Table 10: P6: BERTScore F1.

Model	Full	LoRA	LoRMA	KD
M4	0.4384169273	0.4275381649	0.4158267392	0.3922031896
M6	0.4494979327	0.4422867319	0.4289326157	0.3948127361
M7	**0.4743059657**	0.4678086237	0.4518606025	0.3933845245

Table 11: P7: Micro-F1@7.

Table 12: Secondary metrics for the primary-metric winner in each phase.

Phase	Primary winner	Metrics in column order	First	Second
1	M4	ROUGE-L and BLEU-4	0.188266	0.038554
2	M6 LoRA	Strict macro-P and macro-R	0.575307	0.637822
3	M7 LoRMA	Macro-P and macro-R	0.529999	0.542281
4	M7 Full	Precision and recall	0.818061	0.808399
5	M7 Full	Micro-F1 and subset accuracy	0.505456	0.065289
6	M7 Full	ROUGE-1 and ROUGE-L	0.516457	0.273530
7	M7 Full	Micro-P@7 and micro-R@7	0.4840265220	0.4649681529

Model 7 achieves the best primary score in every phase from P3 to P7. It also wins 18 of 20 comparisons that use the same adaptation method. It leads full fine-tuning, LoRA and LoRMA in all five tasks. Model 6 leads only the P6 and P7 distillation comparisons. The three training replicas produce one 110.10M backbone, so these gains do not require a larger inference ensemble.

The Phase-1 comparison supports the same direction. Model 4 beats Model 6 on BERTScore with fewer parameters and more training tokens per parameter. Model 7 extends this approach to about 4B tokens and 36.3 tokens per parameter. Separate domain training allows each replica to learn local patterns before merging shares those changes across domains. This offers a plausible explanation for its broad transfer performance. Architecture, tokenizer and training budget also changed, so the effect of merging still needs a matched-budget control.

The preferred adaptation depends on the task. Model 7 uses LoRMA for rhetorical roles and full fine-tuning for P4 to P7. For statutes, its macro-F1 advantage over Model 6 is only 0.000240. Model 6 has better micro-F1 at 0.506603 and subset accuracy at 0.066134. For summarization, Model 7 with LoRMA has the best ROUGE-1 at 0.537524, while full fine-tuning leads BERTScore and ROUGE-L. Distillation reduces Model 7 retrieval F1@7 by 0.0809214412 relative to full fine-tuning. Model compression can therefore reduce the advantage of the foundation.

7 Error Analysis and Discussion

7.1 Phase-wise failure modes

Phase 1 often stopped too early. Only 156 of 1,000 continuations reached 128 words, and the median was 84 words. One output states "10.10.2009, I was present before me", which changes the speaker inconsistently. Repeated four-grams occurred at least four times in 133 outputs. This count also includes normal legal phrases, so it flags candidates for review rather than measuring hallucination directly.

NER correctly extracts "SUPREME COURT OF INDIA" at character interval $[18, 40)$ in case `1128835`. However, local DATE F1 is 0.961 while RESP F1 is 0.055. Party roles are harder than dates because the same organization can appear in different roles. STAT has 27 boundary errors and PREC has 19. Even a gold-tag round trip reaches only 0.9916 strict macro-F1, showing a small alignment ceiling. Exact boundaries remain important even when the entity type is clear.

Rhetorical prediction produces no PrecedentOverruled labels. With only three training examples, this class needs closer recall checks. Outcome prediction produces 753 positive and 764 negative labels. Balanced totals do not guarantee correct decisions. A likely failure case is a judgment whose lower-court ruling differs from its final outcome, especially when selected windows miss the ending.

Statute predictions range from one to 39 IDs, compared with a training maximum of 31. Some cases can legitimately require many labels, but broad sets also risk false positives. The best macro-F1 model has subset accuracy 0.065289. Recovering part of the applicable set is therefore much easier than recovering the whole set.

Summary lengths range from 492 to 1,300 words, with median 1,241.5. This is long compared with the 638-word training median. Case `5994` identifies the disciplinary-penalty issue but retains OCR fragments such as "speci al" and "discipl i nary". Length control and text cleanup need improvement. In retrieval, precision at seven is 0.4840265220. A likely error is matching shared legal vocabulary without matching the legal issue or use of a precedent.

7.2 What the metrics do not measure

ROUGE and BLEU measure lexical agreement, while BERTScore measures contextual similarity [3, 4, 5]. They do not verify facts, holdings or citations. Strict span-F1 rejects slightly wrong boundaries. Macro-F1 emphasizes rare labels, while micro-F1 emphasizes aggregate decisions. Retrieval F1@7 ignores ordering within the seven returned cases. A firm would also need source-linked explanations and legal review.

7.3 Limitations and what you would do next

The main limitations are restricted compute, short document coverage, limited legal data and the absence of human evaluation. Future experiments should hold Model$\tilde{7}$'s architecture, tokenizer and 4B-token budget fixed while comparing mixed-corpus training with specialist replicas. General capabilities could be developed through pretraining on non-legal sources such as Wikipedia before legal training, or by assigning one Model$\tilde{7}$ replica to non-legal data to study its influence on specialization and consolidation. Random domain partitions and different merge intervals would further test specialization and consolidation. Repeated seeds would assess small score differences. Longer source coverage and better summary length control are also priorities. Finally, a chained system would require evaluating error propagation between tasks.

7.4 Surprising findings

Model size alone did not determine quality. Model 4 scored 0.838973 on Phase-1 BERTScore, above Model 6 at 0.833631. Model 7 then won all five P3–P7 primary metrics. This can be attributed to higher tokens per parameter as well as the novel training method used for Model 7 which allowed different replicas to learn specialized representations and combine them periodically. However, Model 6 remained stronger on two knowledge distillation comparisons due to larger size of the teacher model and on the P5 secondary metrics. Also, LoRA and LoRMA performing better than full fine tuning in phase 2 and 3 respectively shows effectiveness of PEFT methods. The benefit of a stronger foundational model therefore depends on the task, adaptation method and evaluation metric.

8 Conclusion

The project develops seven legal NLP tasks from foundation training through prior case retrieval. Exact NER alignment, sentence context, long-document windows and task-specific calibration were important parts of the system. LoRA performed best for NER, LoRMA for rhetorical roles, and full fine-tuning for the remaining downstream tasks.

Model 7 is the main methodological contribution. Its domain-specific replicas trained separately and merged periodically into one compact foundation. With about 4B training tokens, it achieved every P3–P7 primary-metric win and led 18 of 20 comparisons using the same adaptation method. These results support further study of specialist training and consolidation under a fixed parameter budget. Controlled experiments are still needed to separate the effects of merging, training exposure and architecture.

GitHub repository: https://github.com/Vivek278/legal-intelligence-system

LLM Usage Declaration

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report? Give model name and vendor (e.g., none / ChatGPT-4.x / Gemini x / Claude x / Grok / Copilot / other).

Answer: ChatGPT-6.1 Sol, Claude Sonnet, Gemini Pro

Q2. For each row, mark Yes or No and state the purpose in one short phrase. "Yes" is acceptable only for the uses permitted by the capstone document.

Part	Used LLM?	Purpose of using the LLM
Understanding concepts / papers	Yes	Understanding relevant techniques in paper for my usecase
Ideation / brainstorming at a high level	Yes	Literature review for available techniques
Debugging small ancillary snippets	Yes	Debugging parts I couldn't debug
Plotting / visualization code	Yes	matplotlib syntax
Grammar / writing style of the report	Yes	Post-writeup grammar checks and style improvement
Report structure / section wording	No	
Designing the model architecture	Yes	Literature review for available techniques
Hyperparameter search / tuning	No	
Analysis of results / conclusions	Yes	Analyzed myself, but generated scripts for plotting and generating tables of metrics I decided
Writing the core training / inference code	Yes	Code stubs when needed

Q3. Did you refer to sources other than lecture slides and an LLM? If yes, list them (papers, blogs, GitHub repositories, textbooks, official starter notebook, evaluation scripts). These must also appear in the bibliography when they influenced the method.

Answer: Research Papers (especially on Model Merging and Plasticity-Adjusted Sampling), Youtube Videos, Stanford CS224N lectures, Kaggle community posts

Q4. Apart from any permitted LLM use, what was *your* contribution? In 8–12 sentences describe your own understanding, architecture decisions, implementation, experiments, and analysis. Name at least two things you tried that failed and what you concluded from them.

My primary contribution was independently understanding the underlying concepts and making the architectural, training and experimental decisions throughout the project. I studied the relevant research on Transformer architectures, parameter-efficient fine-tuning, knowledge distillation and model merging to evaluate their suitability for legal data. My most significant contribution was conceiving Model 7 as a novel approach to address the challenges of training on heterogeneous legal corpora. I proposed a domain-specialized application of DiLoCo in which three replicas train on different legal distributions covering Supreme Court judgments, High Court judgments and Acts with Tribunals. Their weights are periodically merged into a single foundation model. This design was motivated by my analysis of corpus statistics which revealed substantial differences in document lengths and vocabulary distributions across legal sources. I independently designed the replica partitioning and model architecture along with the training budget and merging strategy. I also implemented the complete training pipeline and downstream adaptation and inference pipelines for all five tasks. Initially I experimented with different random initializations for the three replicas but this failed to produce effective merging. Further reading on model merging helped me understand that independently initialized networks can learn different internal representations which makes direct weight averaging unreliable. I therefore switched to a common initialization for all replicas. I also found that prompt and prefix tuning performed substantially worse than LoRA for NER while knowledge distillation underperformed full fine-tuning particularly for prior case retrieval. Through controlled experiments and error analysis I developed an understanding of why different architectural and optimization choices performed better or worse. Model 7 ultimately achieved the best primary scores across Phases 3–7 using a single 110.10M-parameter inference backbone.

References

[1] CS781 organizers, *Capstone Project Competition*, IIT Kanpur, 2026. Dataset: https://huggingface.co/datasets/Exploration-Lab/CS-781-Capstone.

[2] A. Joshi, S. Paul, A. Sharma, P. Goyal, S. Ghosh and A. Modi, "IL-TUR: Benchmark for Indian Legal Text Understanding and Reasoning," ACL, 2024. https://arxiv.org/abs/2407.05399.

[3] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," *Text Summarization Branches Out*, pp. 74–81, 2004. https://aclanthology.org/W04-1013/.

[4] K. Papineni, S. Roukos, T. Ward and W.-J. Zhu, "BLEU: a Method for Automatic Evaluation of Machine Translation," ACL, pp. 311–318, 2002. https://aclanthology.org/P02-1040/.

[5] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger and Y. Artzi, "BERTScore: Evaluating Text Generation with BERT," ICLR, 2020. https://arxiv.org/abs/1904.09675.

[6] V. Sanh, L. Debut, J. Chaumond and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," 2019. https://arxiv.org/abs/1910.01108.

[7] A. Vaswani et al., "Attention Is All You Need," NeurIPS, 2017. https://arxiv.org/abs/1706.03762.

[8] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL, 2019. https://arxiv.org/abs/1810.04805.

[9] L. Dong et al., "Unified Language Model Pre-training for Natural Language Understanding and Generation," NeurIPS, 2019. https://arxiv.org/abs/1905.03197.

[10] M. Lewis et al., "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," ACL, 2020. https://arxiv.org/abs/1910.13461.

[11] J. Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," EMNLP, 2023. https://arxiv.org/abs/2305.13245.

[12] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding," 2021. https://arxiv.org/abs/2104.09864.

[13] B. Zhang and R. Sennrich, "Root Mean Square Layer Normalization," NeurIPS, 2019. https://arxiv.org/abs/1910.07467.

[14] J. L. Ba, J. R. Kiros and G. E. Hinton, "Layer Normalization," 2016. https://arxiv.org/abs/1607.06450.

[15] N. Shazeer, "GLU Variants Improve Transformer," 2020. https://arxiv.org/abs/2002.05202.

[16] D. Hendrycks and K. Gimpel, "Gaussian Error Linear Units (GELUs)," 2016. https://arxiv.org/abs/1606.08415.

[17] R. Sennrich, B. Haddow and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," ACL, 2016. https://arxiv.org/abs/1508.07909.

[18] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei and I. Sutskever, "Language Models are Unsupervised Multitask Learners," OpenAI technical report, 2019.

[19] Y. Wu et al., "Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation," 2016. https://arxiv.org/abs/1609.08144.

[20] LegalOne authors, "LegalOne: A Family of Foundation Models for Reliable Legal Reasoning," arXiv:2602.00642, 2026 (introduces Plasticity-Adjusted Sampling). https://arxiv.org/abs/2602.00642.

[21] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," ICLR, 2019. https://arxiv.org/abs/1711.05101.

[22] P. Micikevicius et al., "Mixed Precision Training," ICLR, 2018. https://arxiv.org/abs/1710.03740.

[23] T. Chen, B. Xu, C. Zhang and C. Guestrin, "Training Deep Nets with Sublinear Memory Cost," 2016. https://arxiv.org/abs/1604.06174.

[24] J. Hoffmann et al., "Training Compute-Optimal Large Language Models," NeurIPS, 2022. https://arxiv.org/abs/2203.15556.

[25] A. Douillard et al., "DiLoCo: Distributed Low-Communication Training of Language Models," 2023. https://arxiv.org/abs/2311.08105.

[26] Y. Nesterov, "A method for solving the convex programming problem with convergence rate $O(1/k^2)$," *Soviet Mathematics Doklady*, 27(2):372–376, 1983.

[27] B. McMahan, E. Moore, D. Ramage, S. Hampson and B. A. y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," AISTATS, 2017. https://arxiv.org/abs/1602.05629.

[28] M. Wortsman et al., "Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time," ICML, 2022. https://arxiv.org/abs/2203.05482.

[29] G. Ilharco et al., "Editing Models with Task Arithmetic," ICLR, 2023. https://arxiv.org/abs/2212.04089.

[30] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," 2021. https://arxiv.org/abs/2106.09685.

[31] H. Bihany, S. Patel and A. Modi, "LoRMA: Low-Rank Multiplicative Adaptation for LLMs," Findings of ACL, 2025. https://arxiv.org/abs/2506.07621.

[32] N. Houlsby et al., "Parameter-Efficient Transfer Learning for NLP," ICML, 2019. https://arxiv.org/abs/1902.00751.

[33] J. Pfeiffer et al., "AdapterFusion: Non-Destructive Task Composition for Transfer Learning," EACL, 2021. https://aclanthology.org/2021.eacl-main.39/.

[34] X. L. Li and P. Liang, "Prefix-Tuning: Optimizing Continuous Prompts for Generation," ACL-IJCNLP, 2021. https://arxiv.org/abs/2101.00190.

[35] B. Lester, R. Al-Rfou and N. Constant, "The Power of Scale for Parameter-Efficient Prompt Tuning," EMNLP, 2021. https://aclanthology.org/2021.emnlp-main.243/.

[36] E. Ben Zaken, Y. Goldberg and S. Ravfogel, "BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models," ACL, 2022. https://aclanthology.org/2022.acl-short.1/.

[37] G. Hinton, O. Vinyals and J. Dean, "Distilling the Knowledge in a Neural Network," 2015. https://arxiv.org/abs/1503.02531.

[38] Y. Kim and A. M. Rush, "Sequence-Level Knowledge Distillation," EMNLP, 2016. https://arxiv.org/abs/1606.07947.

[39] L. A. Ramshaw and M. P. Marcus, "Text Chunking using Transformation-Based Learning," *Third Workshop on Very Large Corpora*, 1995. https://arxiv.org/abs/cmp-lg/9505040.

[40] J. Lafferty, A. McCallum and F. Pereira, "Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data," ICML, 2001.

[41] A. J. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," *IEEE Trans. Information Theory*, 13(2):260–269, 1967.

[42] J. Mullenbach, S. Wiegreffe, J. Duke, J. Sun and J. Eisenstein, "Explainable Prediction of Medical Codes from Clinical Text," NAACL, 2018. https://arxiv.org/abs/1802.05695.

[43] A. van den Oord, Y. Li and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," 2018. https://arxiv.org/abs/1807.03748.

[44] K. Spärck Jones, "A statistical interpretation of term specificity and its application in retrieval," *Journal of Documentation*, 28(1):11–21, 1972.

[45] A. Fan, M. Lewis and Y. Dauphin, "Hierarchical Neural Story Generation," ACL, 2018. https://arxiv.org/abs/1805.04833.

[46] A. Holtzman, J. Buys, L. Du, M. Forbes and Y. Choi, "The Curious Case of Neural Text Degeneration," ICLR, 2020. https://arxiv.org/abs/1904.09751.

[47] N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong and R. Socher, "CTRL: A Conditional Transformer Language Model for Controllable Generation," 2019. https://arxiv.org/abs/1909.05858.

[48] R. Paulus, C. Xiong and R. Socher, "A Deep Reinforced Model for Abstractive Summarization," ICLR, 2018. https://arxiv.org/abs/1705.04304.

A Complete phase result tables

These tables give the complete primary and secondary results for each phase. Bold values mark the maximum for each metric. Model 7 was trained after Phase 1 and has no entry in that phase.

A.1 Phase 1

Table 13: Complete Phase 1 results. Bold marks the best value for each metric.

Model	Method	BERTScore F1	ROUGE-L F1	BLEU-4
M1	Pretrain	0.831894	0.191080	0.043804
M2	Pretrain	0.828659	0.191965	0.040064
M3	Pretrain	0.827758	**0.194339**	**0.046634**
M4	Pretrain	**0.838973**	0.188266	0.038554
M5	Pretrain	0.833433	0.191161	0.043302
M6	Pretrain	0.833631	0.193479	0.044605

A.2 Phase 2

Table 14: Complete Phase 2 results. Bold marks the best value for each metric.

Model	Method	Strict Macro-F1	Strict Macro-Precision	Strict Macro-Recall
M6	Full	0.548211	0.548731	0.547692
M6	LoRA	**0.597164**	0.575307	**0.637822**
M6	LoRMA	0.563143	0.579659	0.564125
M6	KD	0.496312	0.538687	0.489381
M4	Full	0.507839	0.539832	0.499284
M4	LoRA	0.529345	0.545308	0.528961
M4	LoRMA	0.525853	**0.601757**	0.497326
M4	KD	0.494830	0.510329	0.498396

A.3 Phase 3

Table 15: Complete Phase 3 results. Bold marks the best value for each metric.

Model	Method	Macro-F1	Macro-Precision	Macro-Recall
M7	LoRMA	**0.525001**	0.529999	**0.542281**
M7	LoRA	0.513997	0.554742	0.530990
M7	Full	0.505800	0.552000	0.535000
M7	KD	0.493800	0.550000	0.542000
M6	LoRMA	0.522814	0.531605	0.538095
M6	LoRA	0.513336	**0.566227**	0.516802
M6	Full	0.499972	0.554456	0.528415
M6	KD	0.492541	0.518775	0.519101
M4	LoRMA	0.517230	0.527803	0.528280
M4	LoRA	0.509134	0.545275	0.533949
M4	Full	0.490116	0.500390	0.511943
M4	KD	0.489623	0.548421	0.541299

A.4 Phase 4

Table 16: Complete Phase 4 results. Bold marks the best value for each metric.

Model	Method	Macro-F1	Precision	Recall
M7	Full	**0.813447**	**0.818061**	0.808399
M7	LoRA	0.805862	0.786064	**0.843832**
M7	LoRMA	0.800110	0.787955	0.824147
M7	KD	0.795173	0.790731	0.799665
M6	Full	0.798526	0.801347	0.795724
M6	LoRA	0.791384	0.774921	0.808562
M6	LoRMA	0.786927	0.777613	0.796349
M6	KD	0.781463	0.778294	0.784658
M4	Full	0.793281	0.796714	0.789877
M4	LoRA	0.786215	0.769483	0.803690
M4	LoRMA	0.781846	0.772638	0.791179
M4	KD	0.776592	0.773418	0.779792

A.5 Phase 5

Table 17: Complete Phase 5 results. Bold marks the best value for each metric.

Model	Method	Macro-F1	Micro-F1	Subset Accuracy
M7	Full	**0.412709**	0.505456	0.065289
M7	LoRA	0.404459	0.495659	0.052155
M7	LoRMA	0.403452	0.494486	0.048852
M7	KD	0.398731	0.489275	0.043617
M6	Full	0.412469	**0.506603**	**0.066134**
M6	LoRA	0.403817	0.494823	0.050936
M6	LoRMA	0.402614	0.493671	0.047528
M6	KD	0.397926	0.488416	0.042391
M4	Full	0.406448	0.497815	0.055918
M4	LoRA	0.397638	0.486392	0.044217
M4	LoRMA	0.396284	0.485137	0.041863
M4	KD	0.391752	0.479826	0.037294

A.6 Phase 6

Table 18: Complete Phase 6 results. Bold marks the best value for each metric.

Model	Method	BERTScore F1	ROUGE-1	ROUGE-L
M7	Full	**0.850100**	0.516457	**0.273530**
M7	LoRMA	0.845170	**0.537524**	0.269197
M7	LoRA	0.835926	0.421638	0.224571
M7	KD	0.826851	0.335750	0.186306
M6	Full	0.843672	0.487315	0.261842
M6	LoRMA	0.839214	0.472681	0.255936
M6	LoRA	0.832547	0.389426	0.213758
M6	KD	0.827541	0.335053	0.186239
M4	Full	0.837233	0.378011	0.218468
M4	LoRMA	0.831684	0.342716	0.195827
M4	LoRA	0.826392	0.309548	0.174291
M4	KD	0.821853	0.277883	0.156663

A.7 Phase 7

Table 19: Complete Phase 7 results. Bold marks the best value for each metric.

Model	Method	Micro-F1@7	Micro-Precision@7	Micro-Recall@7
M7	Full	**0.4743059657**	**0.4840265220**	**0.4649681529**
M7	LoRA	0.4678086237	0.4773960217	0.4585987261
M7	LoRMA	0.4518606025	0.4611211573	0.4429646786
M7	KD	0.3933845245	0.4014466546	0.3856398379
M6	Full	0.4494979327	0.4587100663	0.4406485235
M6	LoRA	0.4422867319	0.4513728462	0.4335609174
M6	LoRMA	0.4289326157	0.4381062948	0.4201357826
M6	KD	0.3948127361	0.4032718594	0.3867014258
M4	Full	0.4384169273	0.4479026158	0.4293257816
M4	LoRA	0.4275381649	0.4367148295	0.4187426931
M4	LoRMA	0.4158267392	0.4249136857	0.4072679418
M4	KD	0.3922031896	0.4002411091	0.3844817603

B Foundation architectures and additional settings

B.1 Architecture comparison

Table 20 compares the seven foundation designs. Models 1 to 6 were developed during Phase 1. Model 7 was trained after Phase 1 ended and became the strongest foundation for P3 to P7. Training tokens count total exposure, including repeated passes through a corpus.

Table 20: Foundation architectures and training exposure. Ratios are approximate tokens per parameter. Q and KV denote query and key-value heads.

Model	Family	Layers	Width	Q and KV	FFN	Params (M)	Tokens	Tokens per param.
M1	Causal MHA	12	768	12/12	2,048	109.53	1B	9.13
M2	Encoder–decoder	8+4	768	12/12	2,048	118.97	500M	4.20
M3	Prefix LM	12	768	12/12	2,048	109.53	500M	4.57
M4	Causal GQA	16	768	12/2	2,048	122.12	1.5B	12.28
M5	Causal MHA	12	1,024	16/16	3,200	201.09	1B	4.97
M6	Causal GQA	20	1,024	16/4	2,432	234.66	1B	4.26
M7	BERT prefix	12	768	12/12	3,072	110.10	≈4B	≈ **36.3**

Models 1 to 6 use RoPE, RMSNorm and SwiGLU with tied 32K embeddings. Models 1, 3, 4, 5 and 6 use a 1,024-token context. Model 2 has eight encoder layers and four decoder layers. Its encoder limit is 1,088 tokens and its decoder limit is 384 tokens. Model 7 follows a BERT-style architecture [8] with post-LayerNorm, GELU, 512 learned positions and 30,522 uncased WordPieces. Its prefix mask follows the idea of unified language modeling [9]. The foundation weights are trained from scratch.

For Model 4, let $V = 32{,}000$, $d = 768$, $f = 2{,}048$, $d_{kv} = 128$ and $L = 16$. Counting the tied vocabulary matrix once gives

$$N = Vd + L(2d^2 + 2dd_{kv} + 3df + 2d) + d = 122{,}118{,}912.$$

GQA shares two KV heads across 12 query heads [11]. This reduces KV storage while keeping more query projections.

B.2 Training objectives and replica merging

Model 1 uses Plasticity-Adjusted Sampling. It groups documents by length and changes their sampling weights with the learning rate. The target proportions for short, medium and long documents are 20%, 40% and 40%. Model 2 combines denoising and continuation objectives. Model 3 computes next-token loss only on the continuation. Models 4, 5 and 6 use causal next-token prediction.

Model 4 uses AdamW betas $(0.9, 0.95)$, FP16 and gradient checkpointing. Its 5M-token validation set is evaluated every 500 steps. Checkpoints at the same interval allow training to resume across sessions.

Model 7 assigns Supreme Court judgments, High Court judgments, and Acts with Tribunals to three replicas. The shared initial weights align the replicas before local learning. After 500 local AdamW steps, let θ_k^r be the weights of replica k and θ_g^{r-1} the previous merged weights. The

DiLoCo update [25] is

$$\Delta_k^r = \theta_g^{r-1} - \theta_k^r, \qquad \bar{\Delta}^r = \tfrac{1}{3}\sum_{k=1}^{3} \Delta_k^r,$$
$$v^r = \mu v^{r-1} + \bar{\Delta}^r, \qquad \theta_g^r = \theta_g^{r-1} - \eta_{\text{out}}(\bar{\Delta}^r + \mu v^r).$$

Here v is outer momentum and η_{out} is its learning rate. The merged weights begin the next round. Replica optimizer states remain separate. The result is one model for inference. The approximately 4B-token budget covers all replicas together. Exposure ratios depend on the tokenizer and should not be interpreted as equal compute.

B.3 Additional adaptation settings

NER experiments used Houlsby bottlenecks of width 32 and Pfeiffer bottlenecks of width 64 [32, 33]. Prefix tuning added 32 KV slots per layer [34]. Prompt tuning added 32 trainable input tokens [35]. BitFit updated biases [36]. Partial tuning updated the final six Transformer layers. The NER distillation student retained ten layers.

Rhetorical and judgment distillation combine weighted cross-entropy with teacher KL loss. Temperature is 2 and each loss has weight 0.5. The student retains half the layers. Statute distillation uses temperature-scaled Bernoulli targets with BCE. Summarization instead learns teacher-generated sequences alongside references. Retrieval matches teacher embeddings and score distributions in addition to InfoNCE.

Retrieval represents a long case with early, middle and late windows. Text above 150,000 characters is clipped to its beginning and ending. TF-IDF uses up to 60,000 characters, sublinear term frequency, English stopwords and at most 300,000 features. Scores are standardized per query before blending. The lexical weight is selected on validation data. Known alternative positives are excluded from negative targets.

B.4 Document length distribution

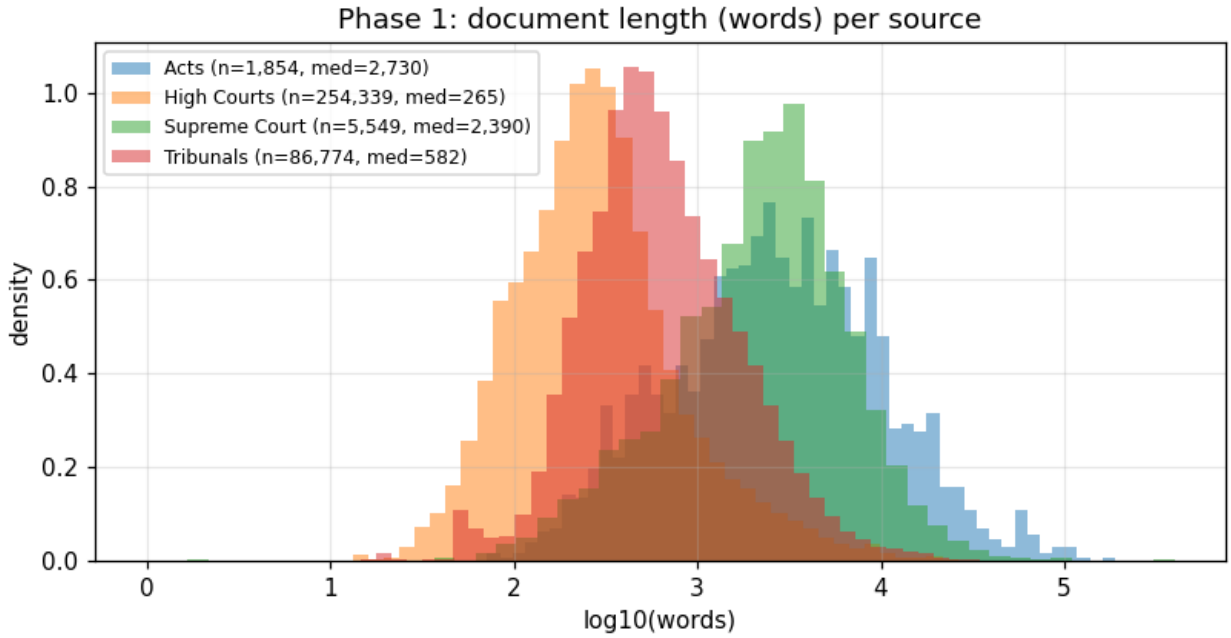


Figure 5: Lengths of nonempty documents in a 10% pretraining sample. Acts and Supreme Court judgments have longer typical documents than High Court records. All four sources have long tails.

C Additional local experiments

C.1 NER adaptation and class performance

The additional NER methods use the same 62 training documents and eight validation documents. Distillation and the Pfeiffer adapter approach LoRA's local F1 of 0.5315. Their small differences require a larger validation sample before drawing a reliable ranking. The emission ensemble reaches 0.5215 and does not improve on its strongest members.

Table 21: Additional local NER comparisons on eight validation documents.

Method	Precision	Recall	F1	Trainable M	Minutes
kd	0.5327	0.5610	0.5312	133.74	34.3
pfeiffer	0.5448	0.5368	0.5297	2.67	11.0
ensemble	0.5555	0.5135	0.5215	0.00	0.0
houlsby	0.4962	0.5246	0.4958	2.69	13.6
bitfit	0.4853	0.5068	0.4869	0.24	22.8
prompt	0.4830	0.3947	0.4172	0.06	26.1
prefix	0.4039	0.4136	0.3999	0.35	26.9

Bottleneck adapters use learning rate 10^{-3}. Prefix tuning uses 2×10^{-3} and prompt tuning uses 5×10^{-3}. BitFit uses 10^{-3} and distillation uses 5×10^{-5}. The head learning rate is 5×10^{-4}. Most methods allow ten epochs. Prompt and prefix tuning allow twelve. Patience is three epochs and warmup is 6%.

Table 22: NER performance by entity type on the local validation set.

Type	Precision	Recall	F1	TP	FP	FN
APP	0.323	0.122	0.177	10	21	72
RESP	0.120	0.035	0.055	3	22	82
A.COUNSEL	0.667	0.381	0.485	8	4	13
R.COUNSEL	0.294	0.417	0.345	5	12	7
JUDGE	0.696	0.762	0.727	16	7	5
WIT	0.684	0.464	0.553	13	6	15
AUTH	0.598	0.603	0.601	73	49	48
COURT	0.898	0.876	0.887	106	12	15
STAT	0.850	0.866	0.858	233	41	36
PREC	0.382	0.667	0.486	60	97	30
DATE	0.940	0.983	0.961	173	11	3
CASENO	0.357	0.185	0.244	10	18	44

DATE and COURT are much stronger than the party-role labels. RESP has 82 false negatives and APP has 72 in the strict evaluation. Boundary analysis also identifies 27 STAT errors, 19 PREC errors and 14 CASENO errors. Boundary categories overlap with strict false positives and false negatives, so they are not added as separate error totals.

C.2 Continuation decoding

The following comparison holds the tokenizer fixed and evaluates each decoder on the same 64 documents. Polishing adds continued pretraining and continuation fine-tuning. Sampling the polished model raises BERTScore but lowers BLEU compared with sampling the original model. This shows why decoder and checkpoint choice depend on the target metric.

Table 23: Continuation decoding on 64 validation documents. Local scores are rounded to six decimal places.

Model and decoder	BERTScore F1	ROUGE-L F1	BLEU
Original greedy	0.812482	0.200492	0.056606
Original greedy, penalty 1.05	0.813853	0.193848	0.055240
Original sampling	0.825715	0.196533	0.059869
Polished greedy	0.813343	0.203300	0.061148
Polished greedy, penalty 1.05	0.818434	0.200073	0.055246
Polished sampling	0.830290	0.198853	0.058406

Selection retained the original sampling system. On a separate 64-document evaluation, it achieved approximately 0.830275 BERTScore, 0.207642 ROUGE-L and 0.063511 BLEU. All 64 outputs reached 128 words. The Phase-1 competition outputs still showed shorter continuations, as discussed in Section 7. Length control therefore needs testing across more documents.